



MACROBUS

Script SQL de Creation de la Base de Donnees

Schema operationnel (OLTP) & Schema analytique (OLAP / Star Schema)

Genere le : 20/06/2026 a 00:55
MacroBus - Vente de Vehicules en Afrique

01 - Schema Operationnel (OLTP)

Creation des 7 tables operationnelles (OLTP) de l'entreprise MacroBus : Territoires, Filiales, Commerciaux, Categories_Vehicule, Vehicules, Commandes et Lignes_Commande. Ces tables enregistrent les operations quotidiennes de vente de vehicules.

Fichier	schema.sql
Taille	1 705 octets
Lignes SQL	47

```
CREATE TABLE IF NOT EXISTS Territoires (  
  id_territoire INT PRIMARY KEY AUTO_INCREMENT,  
  nom_territoire VARCHAR(50) NOT NULL,  
  pays VARCHAR(50) NOT NULL  
);  
CREATE TABLE IF NOT EXISTS Filiales (  
  id_filiale INT PRIMARY KEY AUTO_INCREMENT,  
  nom_filiale VARCHAR(100) NOT NULL,  
  ville VARCHAR(50) NOT NULL,  
  id_territoire INT,  
  FOREIGN KEY (id_territoire) REFERENCES Territoires(id_territoire)  
);  
CREATE TABLE IF NOT EXISTS Commerciaux (  
  id_commercial INT PRIMARY KEY AUTO_INCREMENT,  
  nom VARCHAR(50) NOT NULL,  
  prenom VARCHAR(50) NOT NULL,  
  id_filiale INT,  
  FOREIGN KEY (id_filiale) REFERENCES Filiales(id_filiale)  
);  
CREATE TABLE IF NOT EXISTS Categories_Vehicule (  
  id_categorie INT PRIMARY KEY AUTO_INCREMENT,  
  libelle_categorie VARCHAR(50) NOT NULL  
);  
CREATE TABLE IF NOT EXISTS Vehicules (  
  id_vehicule INT PRIMARY KEY AUTO_INCREMENT,  
  code_vehicule VARCHAR(20) NOT NULL,  
  nom_vehicule VARCHAR(100) NOT NULL,  
  prix_unitaire DECIMAL(10,2) NOT NULL,  
  id_categorie INT,  
  FOREIGN KEY (id_categorie) REFERENCES Categories_Vehicule(id_categorie)  
);  
CREATE TABLE IF NOT EXISTS Commandes (  
  id_commande INT PRIMARY KEY AUTO_INCREMENT,  
  numero_commande VARCHAR(20) NOT NULL,  
  date_commande DATE NOT NULL,  
  id_commercial INT,  
  FOREIGN KEY (id_commercial) REFERENCES Commerciaux(id_commercial)  
);  
CREATE TABLE IF NOT EXISTS Lignes_Commande (  
  id_ligne INT PRIMARY KEY AUTO_INCREMENT,  
  id_commande INT,  
  id_vehicule INT,  
  quantite INT NOT NULL,  
  prix_facture DECIMAL(10,2) NOT NULL,  
  FOREIGN KEY (id_commande) REFERENCES Commandes(id_commande),  
  FOREIGN KEY (id_vehicule) REFERENCES Vehicules(id_vehicule)  
);
```

02 - Schema Analytique (OLAP) + ETL

Creation du schema en etoile (Star Schema) avec 4 dimensions (Dim_Temps, Dim_Vehicule, Dim_Commercial, Dim_Commande) et une table de faits (Fact_Ventes). Inclus le pipeline ETL qui alimente le star schema a partir des donnees operationnelles.

Fichier	star_schema.sql
Taille	4 155 octets
Lignes SQL	104

```
-- =====
-- Star Schema MACROBUS
-- Dimensions (dénormalisées) + Table de Faits
-- =====
-- 1. Dimension Temps
CREATE TABLE IF NOT EXISTS Dim_Temps (
date_complete DATE PRIMARY KEY,
annee INT NOT NULL,
mois INT NOT NULL,
mois_nom VARCHAR(20) NOT NULL,
trimestre INT NOT NULL,
jour INT NOT NULL,
jour_semaine INT NOT NULL,
nom_jour VARCHAR(20) NOT NULL
);
-- 2. Dimension Produit (dénormalise Categorie)
CREATE TABLE IF NOT EXISTS Dim_Vehicule (
id_vehicule INT PRIMARY KEY,
code_vehicule VARCHAR(20) NOT NULL,
nom_vehicule VARCHAR(100) NOT NULL,
prix_unitaire DECIMAL(10,2) NOT NULL,
id_categorie INT,
libelle_categorie VARCHAR(50)
);
-- 3. Dimension Commercial (dénormalise Filiale + Territoire)
CREATE TABLE IF NOT EXISTS Dim_Commercial (
id_commercial INT PRIMARY KEY,
nom VARCHAR(50) NOT NULL,
prenom VARCHAR(50) NOT NULL,
id_filiale INT,
nom_filiale VARCHAR(100),
ville VARCHAR(50),
id_territoire INT,
nom_territoire VARCHAR(50),
pays VARCHAR(50)
);
-- 4. Dimension Commande
CREATE TABLE IF NOT EXISTS Dim_Commande (
id_commande INT PRIMARY KEY,
numero_commande VARCHAR(20) NOT NULL
);
-- 5. Table de Faits Ventes
CREATE TABLE IF NOT EXISTS Fact_Ventes (
id_vente INT PRIMARY KEY AUTO_INCREMENT,
id_commande INT NOT NULL,
id_vehicule INT NOT NULL,
id_commercial INT NOT NULL,
date_complete DATE NOT NULL,
quantite INT NOT NULL,
```

```
prix_facture DECIMAL(10,2) NOT NULL,
```

```
montant_total DECIMAL(10,2) GENERATED ALWAYS AS (quantite * prix_facture) STORED,  
FOREIGN KEY (id_commande) REFERENCES Dim_Commande(id_commande),  
FOREIGN KEY (id_vehicule) REFERENCES Dim_Vehicule(id_vehicule),  
FOREIGN KEY (id_commercial) REFERENCES Dim_Commercial(id_commercial),  
FOREIGN KEY (date_complete) REFERENCES Dim_Temps(date_complete)  
);  
-- =====  
-- ETL : Alimentation du Star Schema  
-- =====  
-- 1. Remplir Dim_Temps à partir des dates des commandes  
INSERT IGNORE INTO Dim_Temps (date_complete, annee, mois, mois_nom, trimestre, jour, jour_semaine, nom_jour)  
SELECT DISTINCT  
c.date_commande,  
YEAR(c.date_commande),  
MONTH(c.date_commande),  
CASE MONTH(c.date_commande)  
WHEN 1 THEN 'Janvier' WHEN 2 THEN 'Février' WHEN 3 THEN 'Mars'  
WHEN 4 THEN 'Avril' WHEN 5 THEN 'Mai' WHEN 6 THEN 'Juin'  
WHEN 7 THEN 'Juillet' WHEN 8 THEN 'Août' WHEN 9 THEN 'Septembre'  
WHEN 10 THEN 'Octobre' WHEN 11 THEN 'Novembre' WHEN 12 THEN 'Décembre'  
END,  
CEIL(MONTH(c.date_commande) / 3.0),  
DAY(c.date_commande),  
DAYOFWEEK(c.date_commande),  
CASE DAYOFWEEK(c.date_commande)  
WHEN 1 THEN 'Dimanche' WHEN 2 THEN 'Lundi' WHEN 3 THEN 'Mardi'  
WHEN 4 THEN 'Mercredi' WHEN 5 THEN 'Jeudi' WHEN 6 THEN 'Vendredi'  
WHEN 7 THEN 'Samedi'  
END  
FROM Commandes c;  
-- 2. Remplir Dim_Vehicule  
INSERT IGNORE INTO Dim_Vehicule  
SELECT  
p.id_vehicule,  
p.code_vehicule,  
p.nom_vehicule,  
p.prix_unitaire,  
p.id_categorie,  
cp.libelle_categorie  
FROM Vehicules p  
LEFT JOIN Categories_Vehicule cp ON p.id_categorie = cp.id_categorie;  
-- 3. Remplir Dim_Commercial  
INSERT IGNORE INTO Dim_Commercial  
SELECT  
c.id_commercial,  
c.nom,  
c.prenom,  
c.id_filiale,  
f.nom_filiale,  
f.ville,  
  
f.id_territoire,  
t.nom_territoire,  
t.pays  
FROM Commerciaux c  
LEFT JOIN Filiales f ON c.id_filiale = f.id_filiale  
LEFT JOIN Territoires t ON f.id_territoire = t.id_territoire;  
-- 4. Remplir Dim_Commande
```

```
INSERT IGNORE INTO Dim_Commande
SELECT id_commande, numero_commande
FROM Commandes;
-- 5. Remplir Fact_Ventes
INSERT INTO Fact_Ventes (id_commande, id_vehicule, id_commercial, date_complete, quantite, prix_facture)
SELECT
c.id_commande,
lc.id_vehicule,
c.id_commercial,
c.date_commande,
lc.quantite,
lc.prix_facture
FROM Lignes_Commande lc
JOIN Commandes c ON lc.id_commande = c.id_commande;
```

03 - Donnees de Test

Jeu de donnees de test representant les activites de MacroBus : 7 territoires, 9 filiales, 16 commerciaux, 8 categories, 18 vehicules, 37 commandes et 52 lignes de commande (2024-2025).

Fichier	data_test.sql
Taille	5 298 octets
Lignes SQL	154

```
-- =====
-- Données de test MACROBUS
-- Entreprise de vente de véhicules
-- =====
-- Territoires
INSERT INTO Territoires (nom_territoire, pays) VALUES
('Afrique de l'Ouest', 'Sénégal'),
('Afrique de l'Ouest', 'Côte d'Ivoire'),
('Afrique Centrale', 'Cameroun'),
('Afrique de l'Est', 'Kenya'),
('Afrique Australe', 'Afrique du Sud'),
('Afrique du Nord', 'Maroc'),
('Afrique du Nord', 'Tunisie');
-- Filiales
INSERT INTO Filiales (nom_filiale, ville, id_territoire) VALUES
('MacroBus Dakar Centre', 'Dakar', 1),
('MacroBus Abidjan Plateau', 'Abidjan', 2),
('MacroBus Douala', 'Douala', 3),
('MacroBus Nairobi', 'Nairobi', 4),
('MacroBus Johannesburg', 'Johannesburg', 5),
('MacroBus Casablanca', 'Casablanca', 6),
('MacroBus Tunis', 'Tunis', 7),
('MacroBus Dakar Almadies', 'Dakar', 1),
('MacroBus Yaoundé', 'Yaoundé', 3);
-- Commerciaux
INSERT INTO Commerciaux (nom, prenom, id_filiale) VALUES
('Diop', 'Amadou', 1),
('Ndiaye', 'Fatou', 1),
('Koné', 'Moussa', 2),
('Kouamé', 'Aya', 2),
('Ewanga', 'Jean-Pierre', 3),
('Biyong', 'Marie', 3),
('Kamau', 'Peter', 4),
('Wanjiku', 'Grace', 4),
('Botha', 'Pieter', 5),
('Nkosi', 'Thandi', 5),
('Benali', 'Hassan', 6),
('El Amrani', 'Leila', 6),
('Ben Salah', 'Mehdi', 7),
('Trabelsi', 'Ines', 7),
('Gueye', 'Ousmane', 8),
('Ndongo', 'Aminata', 9);
-- Categories_Vehicule
INSERT INTO Categories_Vehicule (libelle_categorie) VALUES
('SUV'),
('Berline'),
('Camion'),
('Bus'),
('Utilitaire'),
('Pick-up'),
```

```
('Sport'),
('Électrique');
-- Vehicules (prix en FCFA)
INSERT INTO Vehicules (code_vehicule, nom_vehicule, prix_unitaire, id_categorie) VALUES
('SUV-001', 'MacroBus Terrain X5', 25000000, 1),
('SUV-002', 'MacroBus Terrain X7', 35000000, 1),
('SUV-003', 'MacroBus Terrain X9', 48000000, 1),
('SED-001', 'MacroBus Executive S3', 15000000, 2),
('SED-002', 'MacroBus Executive S5', 22000000, 2),
('SED-003', 'MacroBus Executive S7', 32000000, 2),
('CAM-001', 'MacroBus Cargo T20', 35000000, 3),
('CAM-002', 'MacroBus Cargo T40', 55000000, 3),
('BUS-001', 'MacroBus Transporter 30', 65000000, 4),
('BUS-002', 'MacroBus Transporter 50', 95000000, 4),
('UTI-001', 'MacroBus Utility U100', 8000000, 5),
('UTI-002', 'MacroBus Utility U200', 12000000, 5),
('UTI-003', 'MacroBus Utility U300', 16000000, 5),
('PIK-001', 'MacroBus Ranger R1', 10000000, 6),
('PIK-002', 'MacroBus Ranger R2', 14000000, 6),
('SPO-001', 'MacroBus Speed GT', 42000000, 7),
('ELE-001', 'MacroBus E-Drive E1', 22000000, 8),
('ELE-002', 'MacroBus E-Drive E2', 32000000, 8);
```

```
-- Commandes (2024-2025)
INSERT INTO Commandes (numero_commande, date_commande, id_commercial) VALUES
('CMD-2024-001', '2024-01-15', 1),
('CMD-2024-002', '2024-01-22', 3),
('CMD-2024-003', '2024-02-05', 5),
('CMD-2024-004', '2024-02-18', 7),
('CMD-2024-005', '2024-03-02', 9),
('CMD-2024-006', '2024-03-14', 11),
('CMD-2024-007', '2024-03-28', 13),
('CMD-2024-008', '2024-04-10', 2),
('CMD-2024-009', '2024-04-22', 4),
('CMD-2024-010', '2024-05-06', 6),
('CMD-2024-011', '2024-05-19', 8),
('CMD-2024-012', '2024-06-01', 10),
('CMD-2024-013', '2024-06-15', 12),
('CMD-2024-014', '2024-07-03', 14),
('CMD-2024-015', '2024-07-20', 15),
('CMD-2024-016', '2024-08-05', 1),
('CMD-2024-017', '2024-08-18', 3),
('CMD-2024-018', '2024-09-01', 5),
('CMD-2024-019', '2024-09-14', 7),
('CMD-2024-020', '2024-10-01', 9),
('CMD-2024-021', '2024-10-15', 11),
('CMD-2024-022', '2024-11-01', 13),
('CMD-2024-023', '2024-11-15', 2),
('CMD-2024-024', '2024-12-02', 4),
('CMD-2024-025', '2024-12-16', 6),
('CMD-2025-001', '2025-01-10', 8),
```

```
('CMD-2025-002', '2025-01-24', 10),
('CMD-2025-003', '2025-02-07', 12),
('CMD-2025-004', '2025-02-21', 14),
('CMD-2025-005', '2025-03-05', 15),
('CMD-2025-006', '2025-03-19', 1),
('CMD-2025-007', '2025-04-02', 3),
('CMD-2025-008', '2025-04-16', 5),
('CMD-2025-009', '2025-05-01', 7),
('CMD-2025-010', '2025-05-14', 9),
```

```
('CMD-2025-011', '2025-05-28', 11),
('CMD-2025-012', '2025-06-10', 13);
-- Lignes_Commande (prix facture en FCFA)
INSERT INTO Lignes_Commande (id_commande, id_vehicule, quantite, prix_facture) VALUES
(1, 1, 2, 24000000),
(1, 14, 1, 9500000),
(2, 5, 3, 21000000),
(3, 7, 2, 34000000),
(3, 17, 1, 21000000),
(4, 9, 1, 62000000),
(4, 11, 4, 7500000),
(5, 3, 1, 46000000),
(5, 15, 2, 13500000),
(6, 4, 2, 14500000),
(7, 8, 1, 52000000),
(7, 12, 3, 11500000),
(8, 2, 1, 34000000),
(8, 6, 2, 31000000),
(9, 10, 1, 92000000),
(10, 13, 3, 15500000),
(11, 1, 2, 24500000),
(12, 16, 1, 40000000),
(13, 18, 2, 31000000),
(14, 5, 1, 21500000),
(15, 7, 1, 34500000),
(16, 3, 1, 47000000),
(16, 17, 2, 21500000),
(17, 9, 1, 63000000),
(18, 2, 2, 34500000),
(19, 14, 3, 9500000),
(20, 6, 1, 31500000),
(20, 11, 2, 7500000),
(21, 8, 1, 53000000),
(21, 15, 1, 13500000),
(22, 4, 2, 14000000),
(23, 1, 1, 24800000),
(23, 12, 2, 11800000),
(24, 16, 1, 41000000),
(25, 10, 1, 93000000),
(25, 13, 2, 15500000),
(26, 3, 1, 47500000),
```

```
(27, 5, 2, 21800000),
(28, 18, 1, 31500000),
(28, 14, 2, 9800000),
(29, 7, 1, 34800000),
(30, 9, 1, 64000000),
(30, 17, 1, 21800000),
(31, 2, 2, 34800000),
(31, 11, 3, 7800000),
(32, 6, 1, 31800000),
(32, 15, 2, 13800000),
(33, 8, 1, 53500000),
(34, 1, 2, 24800000),
(35, 4, 1, 14800000),
(35, 12, 2, 11800000),
(36, 16, 1, 41500000);
```


04 - Vues MySQL pour Power BI

7 vues optimisées pour Power BI et les API REST : Ventes_Complete, Ventes_Par_Commercial, Ventes_Par_Mois, Ventes_Par_Categorie, Ventes_Par_Territoire, Top_Vehicules et KPIs_Globaux.

Fichier	powerbi_views.sql
Taille	3 892 octets
Lignes SQL	99

```
-- =====
-- Vues Power BI pour MACROBUS
-- Utilisation : mysql -u root -p MacroBus_Production < powerbi_views.sql
-- =====
-- 1. Vue principale : toutes les ventes avec leurs dimensions
CREATE OR REPLACE VIEW Ventes_Complete AS
SELECT
fv.id_vente,
fv.quantite,
fv.prix_facture,
fv.montant_total,
-- Dimensions temps
dt.date_complete,
dt.annee,
dt.mois,
dt.mois_nom,
dt.trimestre,
dt.jour,
dt.jour_semaine,
dt.nom_jour,
-- Dimensions vehicule
dv.id_vehicule,
dv.code_vehicule,
dv.nom_vehicule,
dv.prix_unitaire AS prix_catalogue,
dv.libelle_categorie AS categorie,
-- Dimensions commercial
dc.id_commercial,
dc.nom AS commercial_nom,
dc.prenom AS commercial_prenom,
CONCAT(dc.prenom, ' ', dc.nom) AS commercial_nom_complet,
dc.nom_filiale AS filiale,
dc.ville,
dc.nom_territoire AS territoire,
dc.pays,
-- Dimensions commande
dco.numero_commande
FROM Fact_Ventes fv
JOIN Dim_Temps dt ON fv.date_complete = dt.date_complete
JOIN Dim_Vehicule dv ON fv.id_vehicule = dv.id_vehicule
JOIN Dim_Commercial dc ON fv.id_commercial = dc.id_commercial
JOIN Dim_Commande dco ON fv.id_commande = dco.id_commande;
-- 2. Vue : CA par commercial
CREATE OR REPLACE VIEW Ventes_Par_Commercial AS
SELECT
dc.id_commercial,
CONCAT(dc.prenom, ' ', dc.nom) AS commercial,
dc.nom_filiale AS filiale,
dc.ville,
dc.pays,
```

```

COUNT(DISTINCT fv.id_commande) AS nb_commandes,
SUM(fv.quantite) AS nb_vehicules,
ROUND(SUM(fv.montant_total), 0) AS ca_total,
ROUND(AVG(fv.montant_total), 0) AS panier_moyen
FROM Fact_Ventes fv
JOIN Dim_Commercial dc ON fv.id_commercial = dc.id_commercial
GROUP BY dc.id_commercial, commercial, dc.nom_filiale, dc.ville, dc.pays;
-- 3. Vue : CA par mois
CREATE OR REPLACE VIEW Ventes_Par_Mois AS
SELECT
dt.annee,
dt.mois,
dt.mois_nom,
dt.trimestre,
COUNT(DISTINCT fv.id_commande) AS nb_commandes,
SUM(fv.quantite) AS nb_vehicules,
ROUND(SUM(fv.montant_total), 0) AS ca_total
FROM Fact_Ventes fv
JOIN Dim_Temps dt ON fv.date_complete = dt.date_complete
GROUP BY dt.annee, dt.mois, dt.mois_nom, dt.trimestre
ORDER BY dt.annee, dt.mois;
-- 4. Vue : CA par categorie
CREATE OR REPLACE VIEW Ventes_Par_Categorie AS
SELECT
dv.libelle_categorie AS categorie,
COUNT(DISTINCT fv.id_commande) AS nb_commandes,
SUM(fv.quantite) AS nb_vehicules,
ROUND(SUM(fv.montant_total), 0) AS ca_total
FROM Fact_Ventes fv
JOIN Dim_Vehicule dv ON fv.id_vehicule = dv.id_vehicule
GROUP BY dv.libelle_categorie
ORDER BY ca_total DESC;
-- 5. Vue : CA par territoire
CREATE OR REPLACE VIEW Ventes_Par_Territoire AS
SELECT
dc.pays,
dc.nom_territoire AS territoire,
COUNT(DISTINCT fv.id_commande) AS nb_commandes,
ROUND(SUM(fv.montant_total), 0) AS ca_total
FROM Fact_Ventes fv
JOIN Dim_Commercial dc ON fv.id_commercial = dc.id_commercial
GROUP BY dc.pays, dc.nom_territoire
ORDER BY ca_total DESC;
-- 6. Vue : Top vehicules
CREATE OR REPLACE VIEW Top_Vehicules AS
SELECT
dv.nom_vehicule,
dv.libelle_categorie AS categorie,
dv.prix_unitaire AS prix_catalogue,
SUM(fv.quantite) AS total_vendus,

```

```

ROUND(SUM(fv.montant_total), 0) AS ca_total
FROM Fact_Ventes fv
JOIN Dim_Vehicule dv ON fv.id_vehicule = dv.id_vehicule
GROUP BY dv.nom_vehicule, dv.libelle_categorie, dv.prix_unitaire
ORDER BY total_vendus DESC;
-- 7. Vue : KPIs globaux
CREATE OR REPLACE VIEW KPIs_Globaux AS
SELECT
ROUND(SUM(montant_total), 0) AS ca_total,

```

```
COUNT(DISTINCT id_commande) AS nb_commandes,  
SUM(quantite) AS nb_vehicules,  
ROUND(AVG(montant_total), 0) AS panier_moyen,  
(SELECT COUNT(*) FROM Dim_Commercial) AS nb_commerciaux,  
(SELECT COUNT(*) FROM Dim_Vehicule) AS nb_modeles;
```